

Lab Evaluation 1

Roll: 2103064

1(a) Code Snippet:

```
%option noyywrap

%{
    #include "cal.tab.h"
    #include <stdlib.h>
    #include <string.h>
}%

letter [A-Za-z]
digit [0-9]
us [_]
id ({us}|{letter})({us}|{digit}|{letter})*
number {digit}+

%%

" " { }
"main" { printf("MAIN: %s\n",yytext); return MAIN; }
"shuru" { printf("SHURU: %s\n",yytext); return SHURU; }
"of" { printf("OF: %s\n", yytext); return OF; }
"type" { printf("TYPE: %s\n",yytext); return TYPE; }
"int" { printf("INT: %s\n", yytext); return INT; }
"jotokkhon" { printf("JOTOKKHON: %s\n",yytext); return JOTOKKHON; }
}

"print" { printf("PRINT: %s\n",yytext); return PRINT; }
"unit" { printf("UNIT: %s\n", yytext); return UNIT; }
"increase" { printf("INCREASE: %s\n",yytext); return INCREASE; }
"shesh" { printf("SHESH: %s\n",yytext); return SHESH; }

 "(" { printf("LP: %s\n",yytext); return LP; }
 ")" { printf("RP: %s\n",yytext); return RP; }
 "=" { printf("ASSIGN: %s\n",yytext); return ASSIGN; }
 "<" { printf("LT: %s\n",yytext); return LT; }
 "+" { printf("ADD: %s\n",yytext); return ADD; }
 "/" { printf("DIV: %s\n",yytext); return DIV; }
 "-" { printf("SUB: %s\n",yytext); return SUB; }
 "*" { printf("MUL: %s\n",yytext); return MUL; }
```

```

{number} { printf("NUM: %s\n",yytext); yylval=atoi(yytext); return
NUM;}
{id} { printf("ID: %s\n",yytext); return ID; }
\[^\"]*\" { printf("STRING_LITERAL: %s\n", yytext); return
STRING_LITERAL; }
%%

/* int yywrap() {
    return 1;
} */

```

1(a) Output.

```

MAIN: main
SHURU: shuru

ID: a
ASSIGN: =
NUM: 10
OF: of
TYPE: type
INT: int

JOTOKKHON: jotokkhon
ID: a
LT: <
NUM: 20

PRINT: print
ASSIGN: =
STRING_LITERAL: "counting"

ID: b
OF: of
TYPE: type
INT: int
ASSIGN: =
NUM: 20
MUL: *
LP: (

```

NUM: 30

ADD: +

NUM: 10

SUB: -

NUM: 5

RP:)

UNIT: unit

Result-> 400

INCREASE: increase

NUM: 1

MAIN: main

SHESH: shesh

Statements parsed successfully!

1(b) Code Snippet:

```
%{  
  
    #include <stdio.h>  
    #include <stdlib.h>  
    #include <string.h>  
  
    void yyerror(const char *s);  
    int yylex();  
  
    /* Symbol Table (memory) */  
    char sym_names[100][50]; // store 100 variables  
    int sym_values[100];  
    int sym_count = 0;  
    int temp_arr[100];  
    int idx = 0;  
  
    /* Get the variable */  
    int get_variable(char* name) {  
        for(int i = 0; i < sym_count; i++) {  
            if(strcmp(sym_names[i], name) == 0) {  
                return sym_values[i];  
            }  
        }  
    }  
}
```

```

        return 0; // in not found case
    }

    /* set a new variable */
    void set_variable(char* name, int val) {
        // First check already prsent or not
        for(int i = 0; i < sym_count; i++) {
            if(strcmp(sym_names[i], name) == 0) {
                sym_values[i] = val;
                return;
            }
        }
        // O/W add new variable
        strcpy(sym_names[sym_count], name);
        sym_values[sym_count] = val;
        sym_count++;
    }

%}

%token MAIN SHURU OF TYPE INT JOTOKKHON PRINT UNIT INCREASE SHESH
%token LP RP ASSIGN LT ADD DIV SUB MUL NUM ID STRING_LITERAL

%left ADD SUB
%left MUL DIV
%right ASSIGN

%%

Program: MAIN SHURU statements MAIN SHESH
        ;

statements: statements statement
        |
        ;

statement: ID ASSIGN NUM OF TYPE INT
        | JOTOKKHON ID LT NUM
        | PRINT ASSIGN STRING_LITERAL

```

```

        | ID OF TYPE INT ASSIGN exp { printf("Result-> %d\n",
$6);}
        | UNIT INCREASE NUM
        ;

exp: LP exp RP
    | exp ADD exp { $$=$1+$3; }
    | exp SUB exp { $$=$1-$3; }
    | exp MUL exp { $$=$1*$3; }
    | exp DIV exp { $$=$1/$3; }
    | NUM { $$= $1; }

%%

void yyerror(const char *s)
{
    fprintf(stderr, "Syntax Error: %s\n", s);
}

int main()
{
    /* printf("Enter the C++ statements to parse:\n"); */
    if (yyparse() == 0) {
        printf("Statements parsed successfully!\n");
    }
    return 0;
}

```

1(b) Output.

Statements parsed successfully!

Makefile:

```
main: cal.y cal.l  
      bison -d cal.y  
      flex cal.l  
      gcc cal.tab.c lex.yy.c  
      a < input.txt > output.txt
```